Registers and Counters

Chapter 06

Digital Design 4th Ed. by

Morris Mano

Registers

- Registers and Counters are types of Sequential Circuits.

- A REGISTER is a group of flip-flops
- An n-bit register consists of n flip-flops, capable of storing n-bits of binary information
- In addition to flip-flops, registers may have additional combinational circuits for performing certain data processing tasks.

Registers

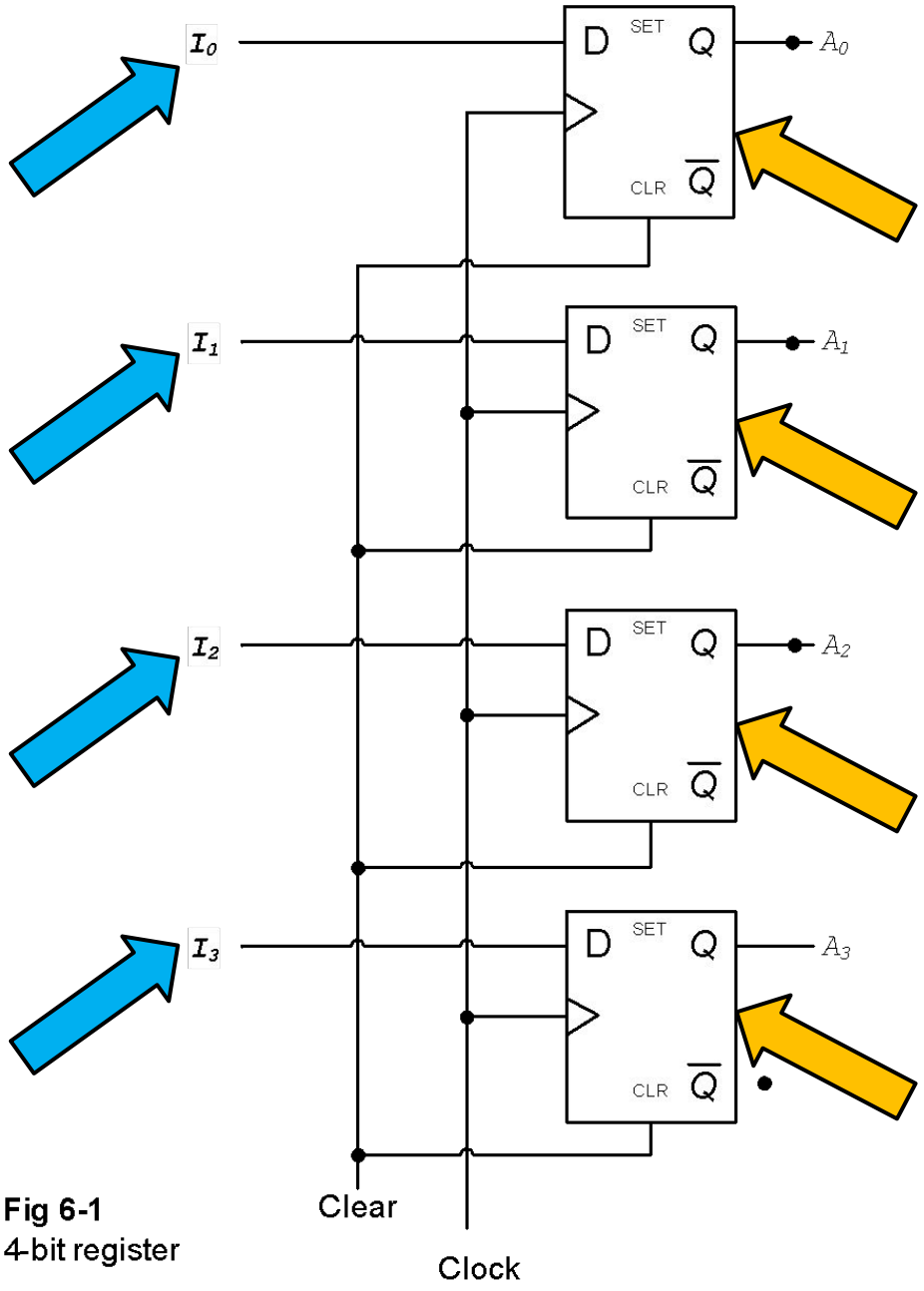
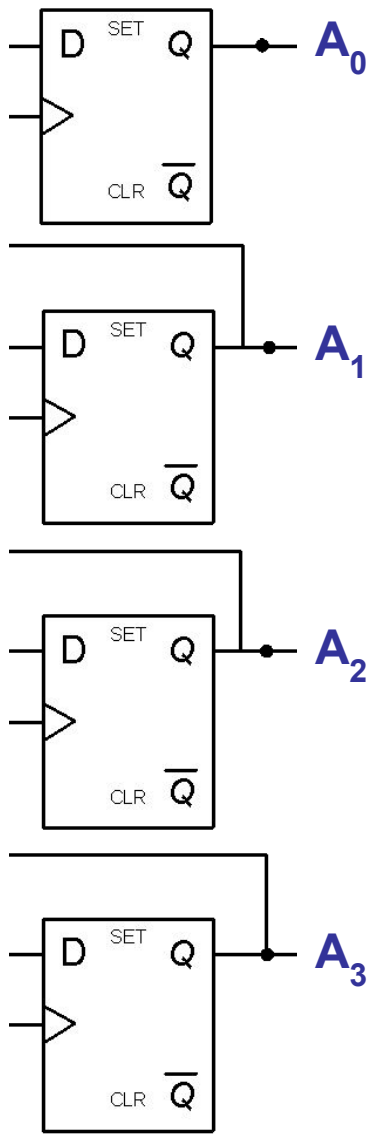


Fig 6-1
4-bit register

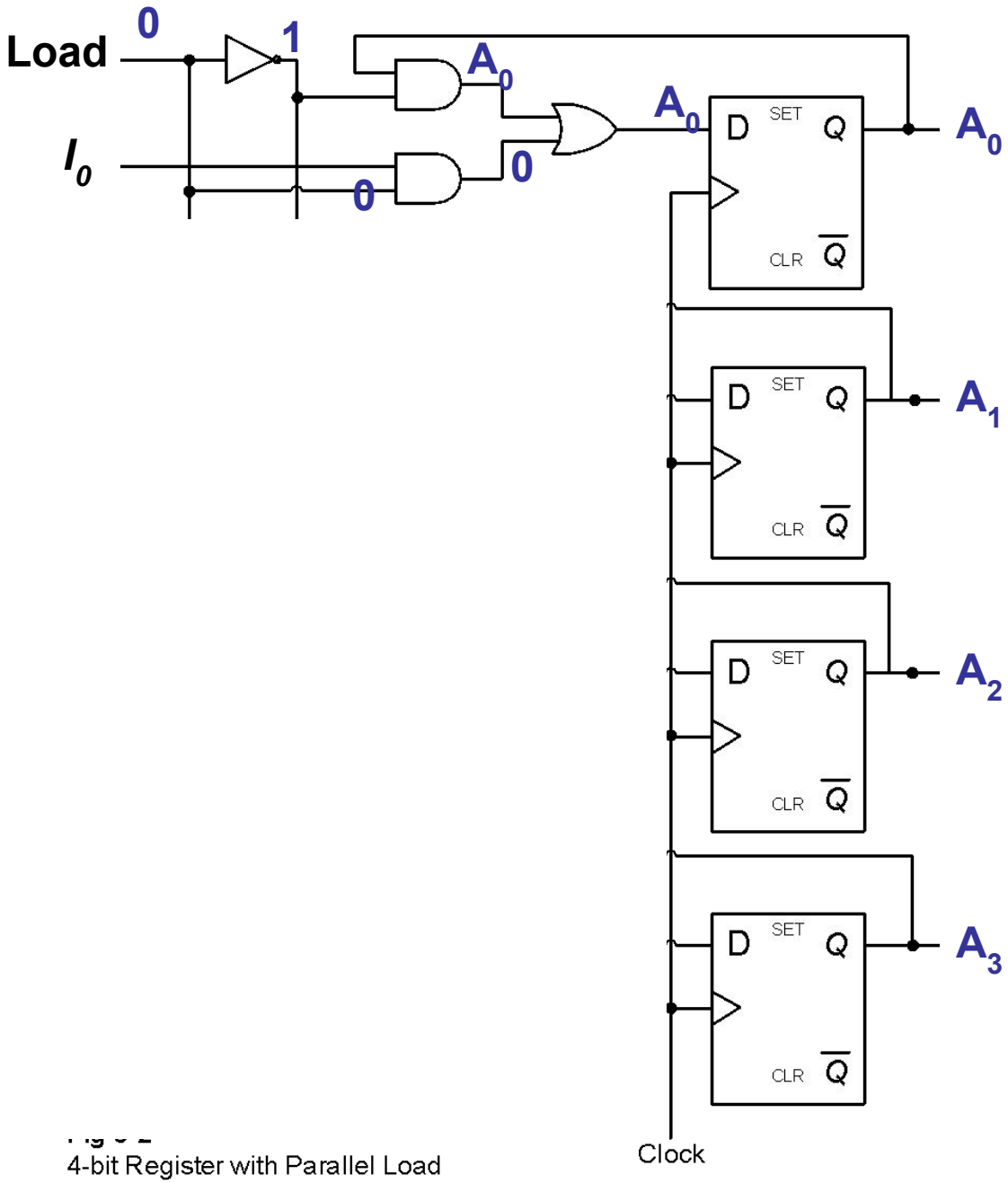
Register with Parallel Load

Load



4-bit Register with Parallel Load

Register with Parallel Load



Load = 0

$D_0 = A_0$

No new data will be stored. Old data (A_0) will be refreshed and retained.

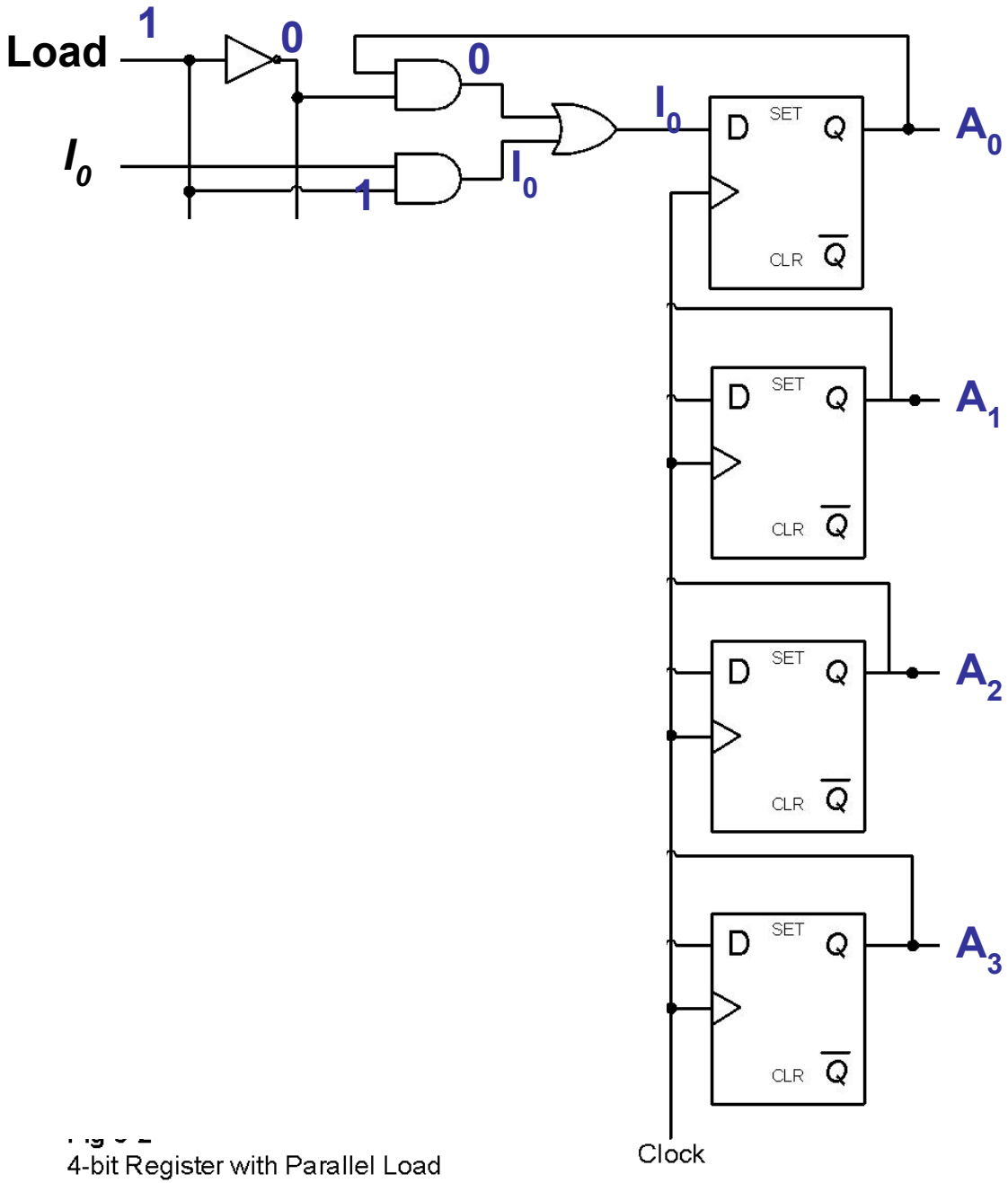
4-bit Register with Parallel Load

Register with Parallel Load

Load = 1

$D_0 = I_0$

New data I_0 will be stored on next clock edge.



4-bit Register with Parallel Load

Shift Register

- A register capable of shifting its binary information in one or both directions.

- Can be used for serial transfers

Shift Register

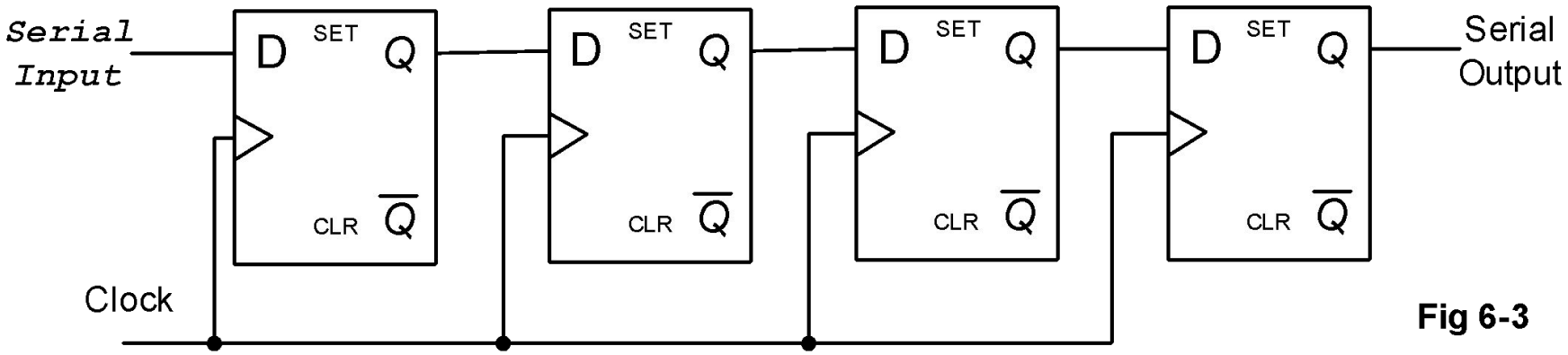


Fig 6-3

Shift Register

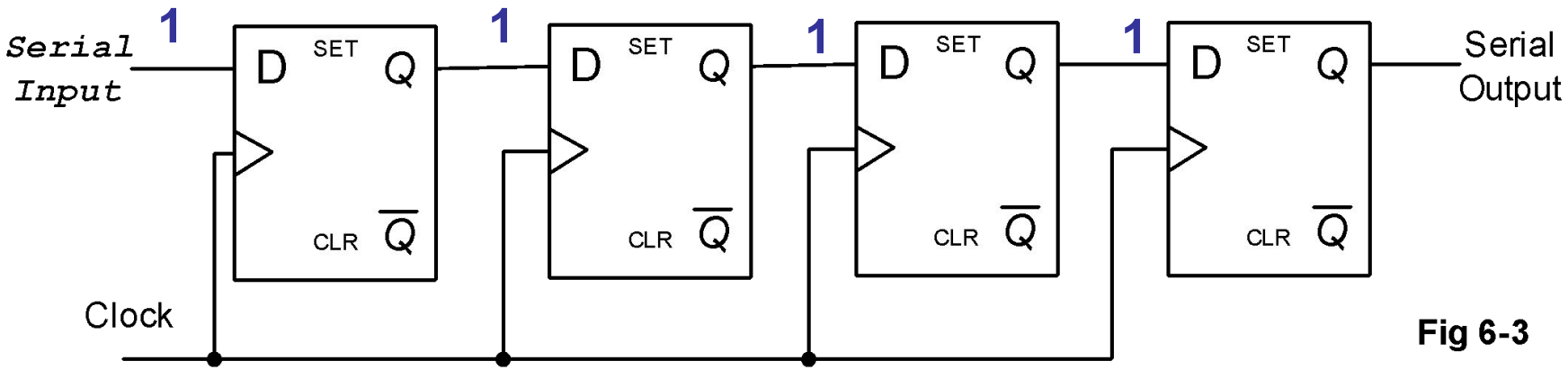
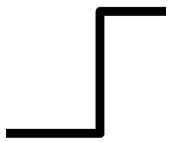


Fig 6-3



Shift Register

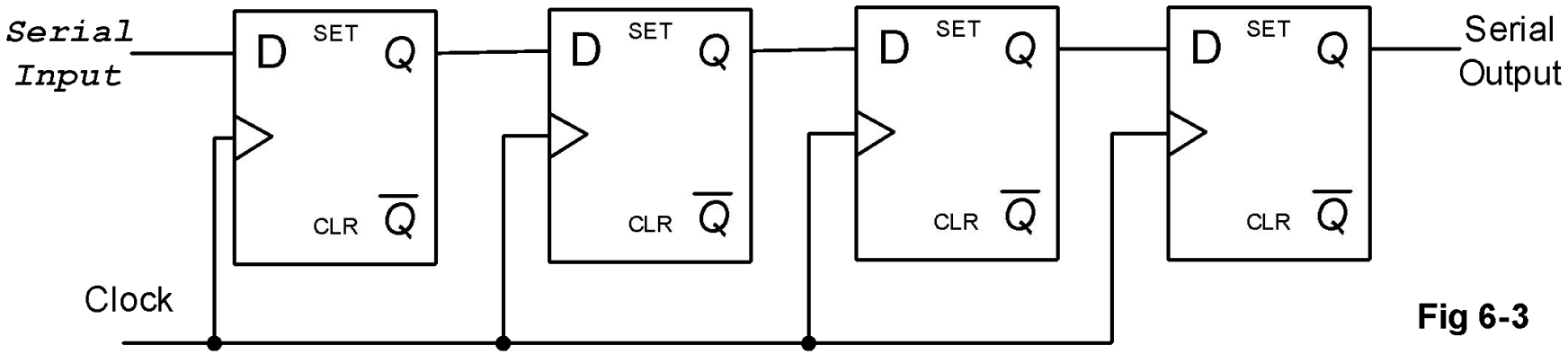
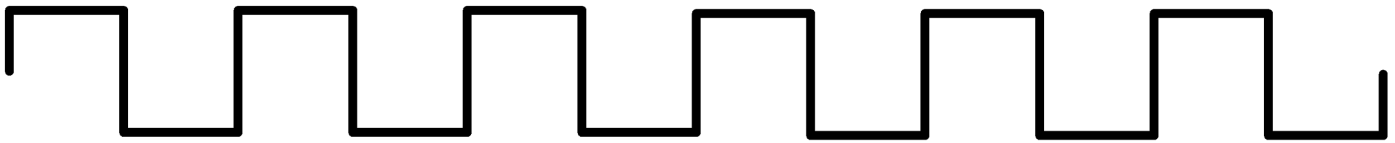


Fig 6-3



COUNTERS

Counters

- A counter is essentially a register that goes through a predetermined sequence states on the application of input.
  - The input can be a clock pulse or any other input.
- The gates in the counter are connected in such a way as to produce the prescribed sequence of binary states.

Counters - Categories

1. Ripple Counter
   - The flip flop output transitions serve as a source of triggering other flip-flops
   - i.e. C input of some or all flip-flops is not triggered by the common clock pulse
2. Synchronous Counter
   - C input of all flip flops is connected to same clock input

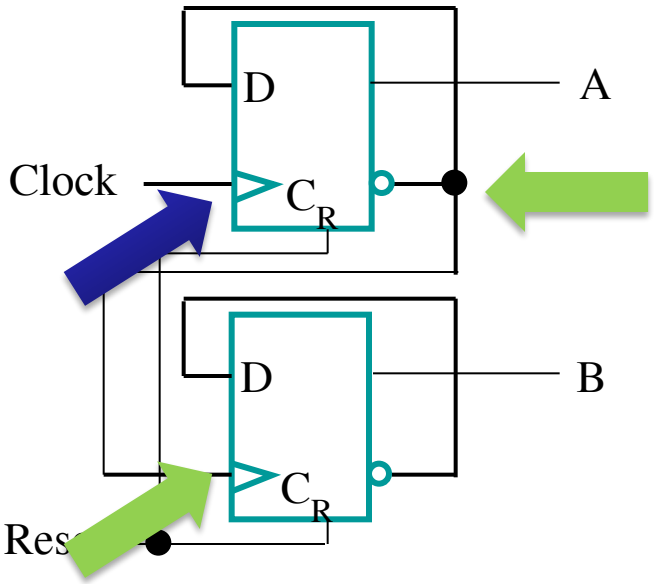
Binary Ripple Counter

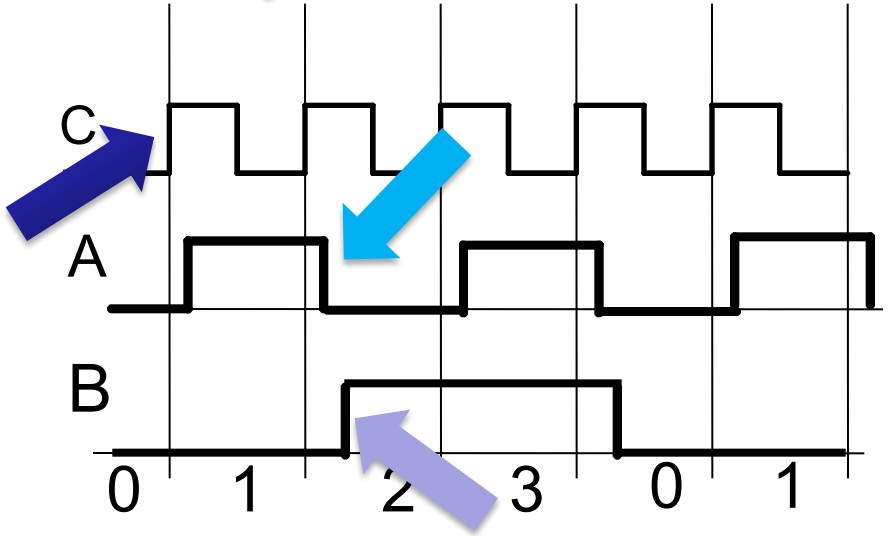
- A counter that follows the binary number sequence is called a BINARY COUNTER.

- A BINARY RIPPLE COUNTER consists of a series connection of complementing flip-flops, with the output of each flip-flop connected to the C input of the next flip-flop.
- The flip-flop holding the least significant bit is given an input clock pulse.

Ripple Counter

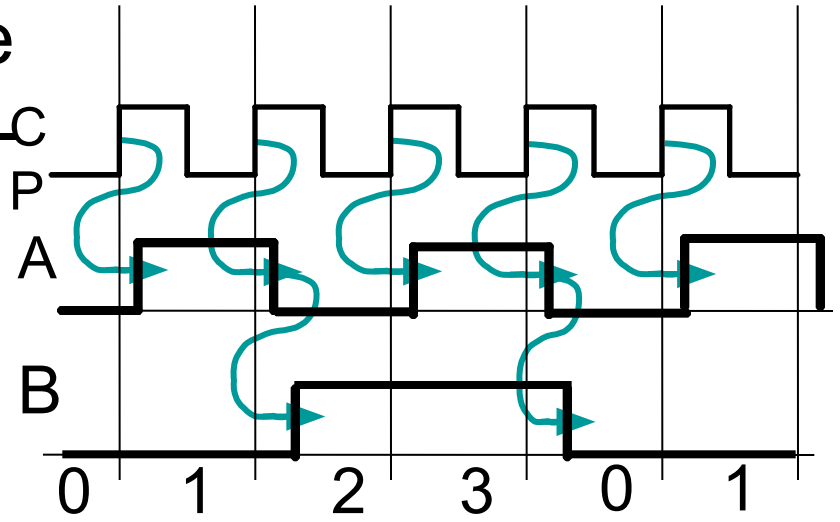
- **How does it work?**
  - When there is a positive edge on the clock input of A, A complements
  - The clock input for flip-flop B is the complemented output of flip-flop A
  - When flip A changes from 1 to 0, there is a positive edge on the clock input of B causing B to complement





Ripple Counter (continued)

- The arrows show the cause-effect relation-ship from the prior slide



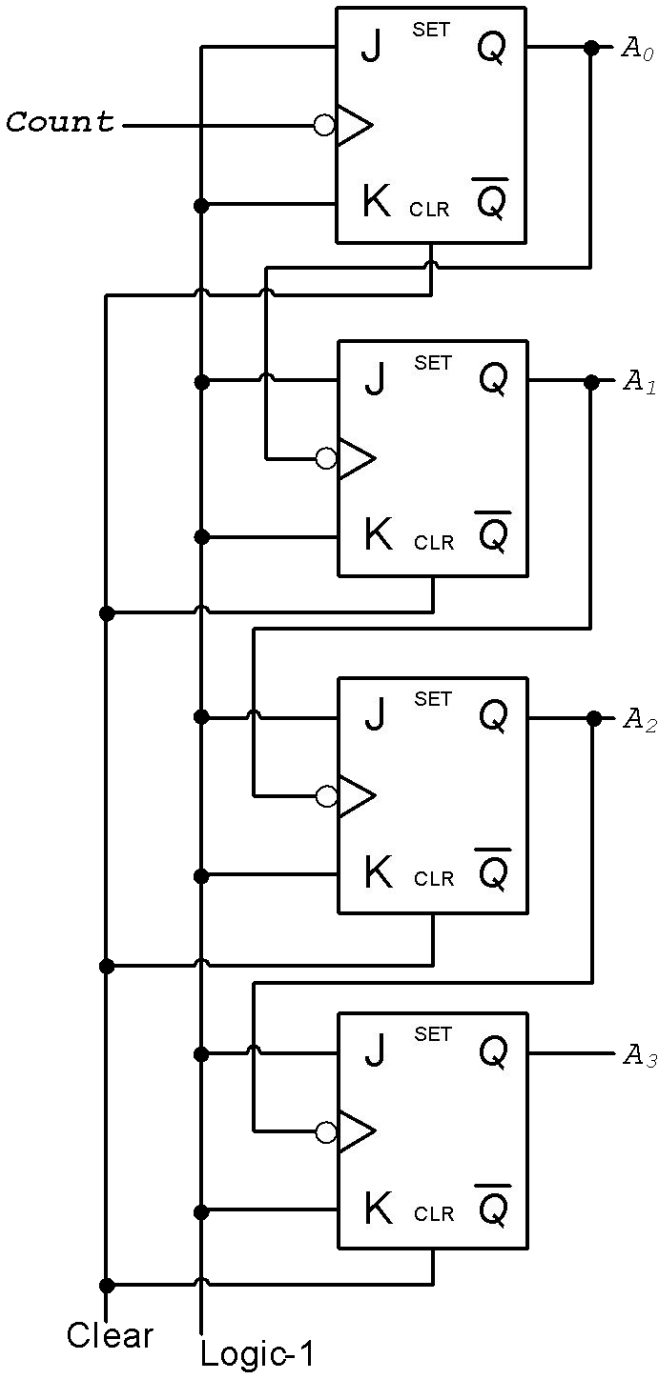
- The corresponding sequence of states (B,A) = (0,0), (0,1) (1,0) (1,1) (0,0) (0,1), …
- Each additional bit, C, D, …behaves like bit B, changing half as frequently as the bit before it.
- For 3 bits: (C,B,A) = (0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1), (1,1,0), (1,1,1), (0,0,0), …

Ripple Counter (continued)

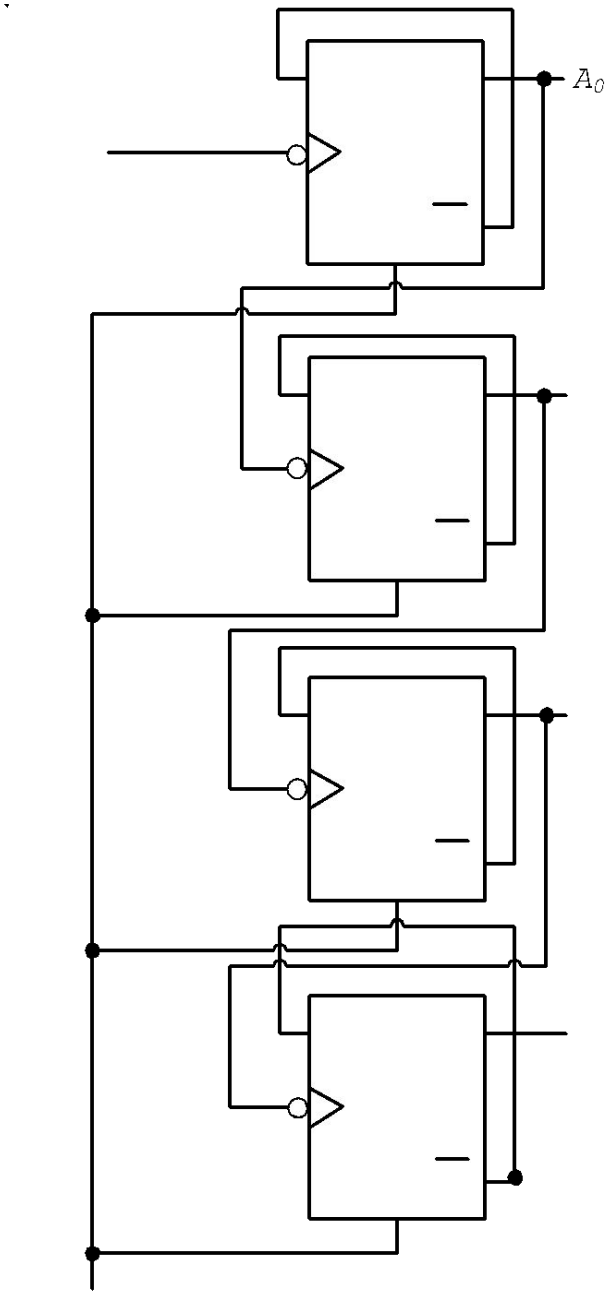
- These circuits are called *ripple counters* because each edge sensitive transition (positive in the example) causes a change in the next flip-flop's state.
- The changes "ripple" upward through the chain of flip-flops, i. e., each transition occurs after a clock-to-output delay from the stage before.
- To see this effect in detail look at the waveforms on the next slide.

Binary Ripple Counter with J-K Flip Flops

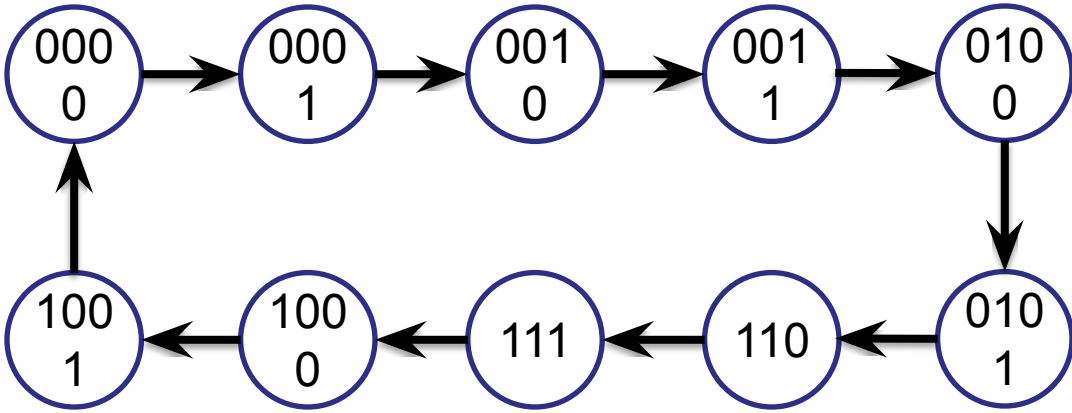
A3	A2	A1	A0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1
0	0	0	0



Binary Ripple Counter



BCD-Ripple Counter



BCD-Ripple Counter

Present State				Next State			
A	B	C	D	A	B	C	D
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	0	0	0	0

BCD-Ripple Counter

A	B	C	D
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1

Present State

Next State

BCD-Ripple Counter

A	B	C	D
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1

Present State

Next State

BCD-Ripple Counter

A	B	C	D
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1

Present State

Next State

BCD-Ripple Counter

A	B	C	D
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1

Present State

Next State

BCD-Ripple Counter

A	B	C	D
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1

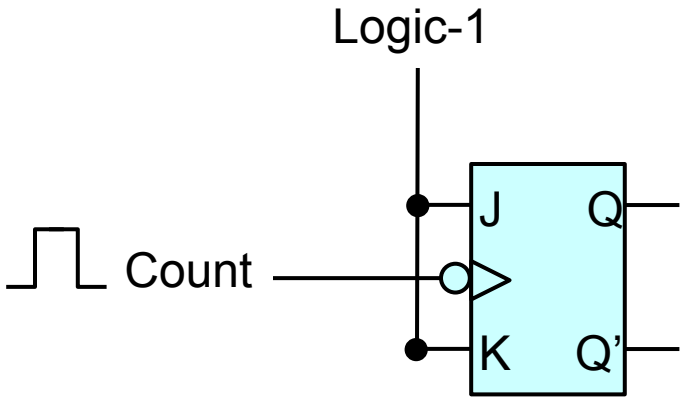
Present State

Next State

BCD-Ripple Counter

A	B	C	D	
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	Present State
0	0	0	0	Next State

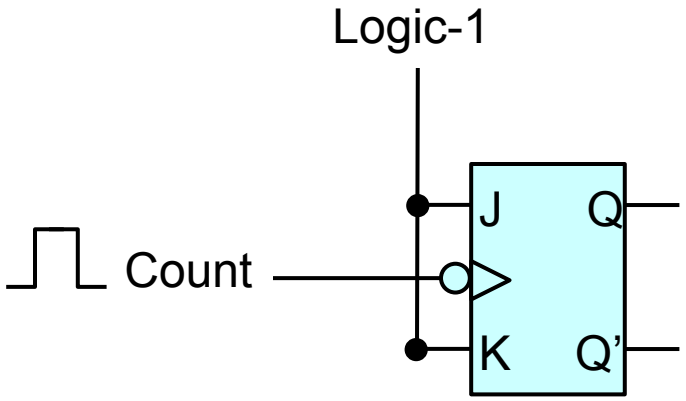
BCD-Ripple Counter



		D
		0
		1
		0
		1
		0
		1
		0
		1
		0
		1
		0

BCD-Ripple Counter

		C	D
		0	0
		0	1
		1	0
		1	1
		0	0
		0	1
		1	0
		1	1
		0	0
		0	1
		0	0



BCD-Ripple Counter

BCD			
Q8	Q4	Q2	Q1
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
0	0	0	0

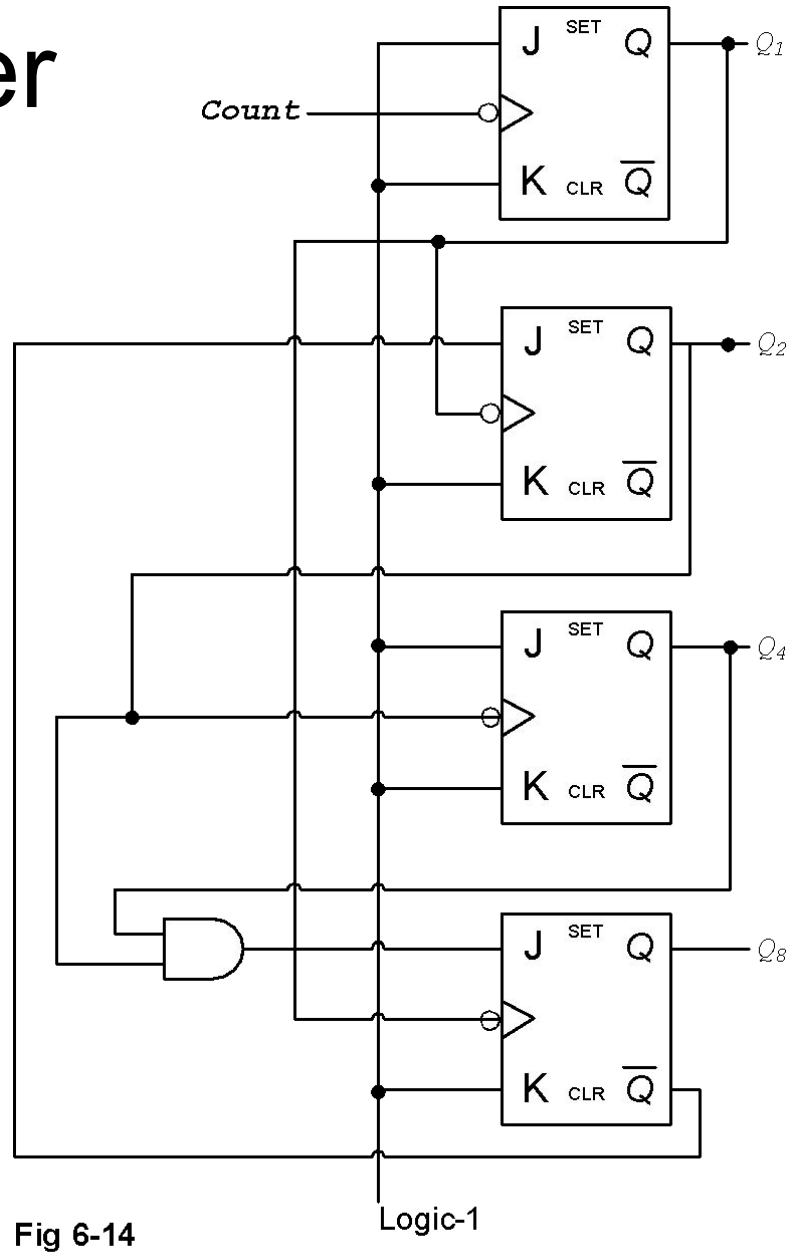


Fig 6-14
BCD ripple counter

BCD-Ripple Counter

BCD			
Q8	Q4	Q2	Q1
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
0	0	0	0

- Q1 triggers FF_2 and FF_8
- K of all are logic 1
- J_1 is logic 1
- J_2 is Q_8'
- J_4 is logic 1
- J_8 is Q_2Q_4

BCD-Ripple Counter

BCD			
Q8	Q4	Q2	Q1
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
0	0	0	0

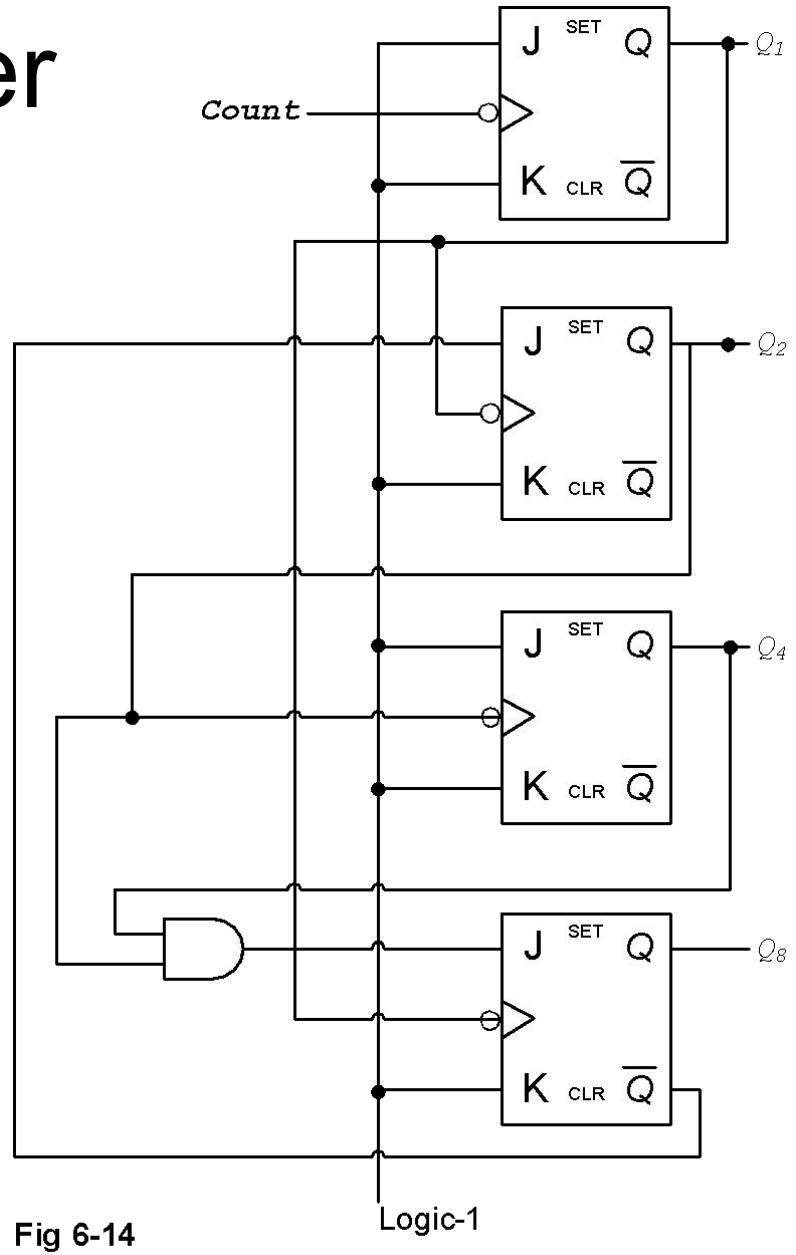


Fig 6-14
BCD ripple counter

BCD-Ripple Counter

- Q1 triggers FF_2 and FF_8

- K of all are logic 1
- J_1 is logic 1
- J_2 is Q_8'

- J_4 is logic 1
- J_8 is Q_2Q_4

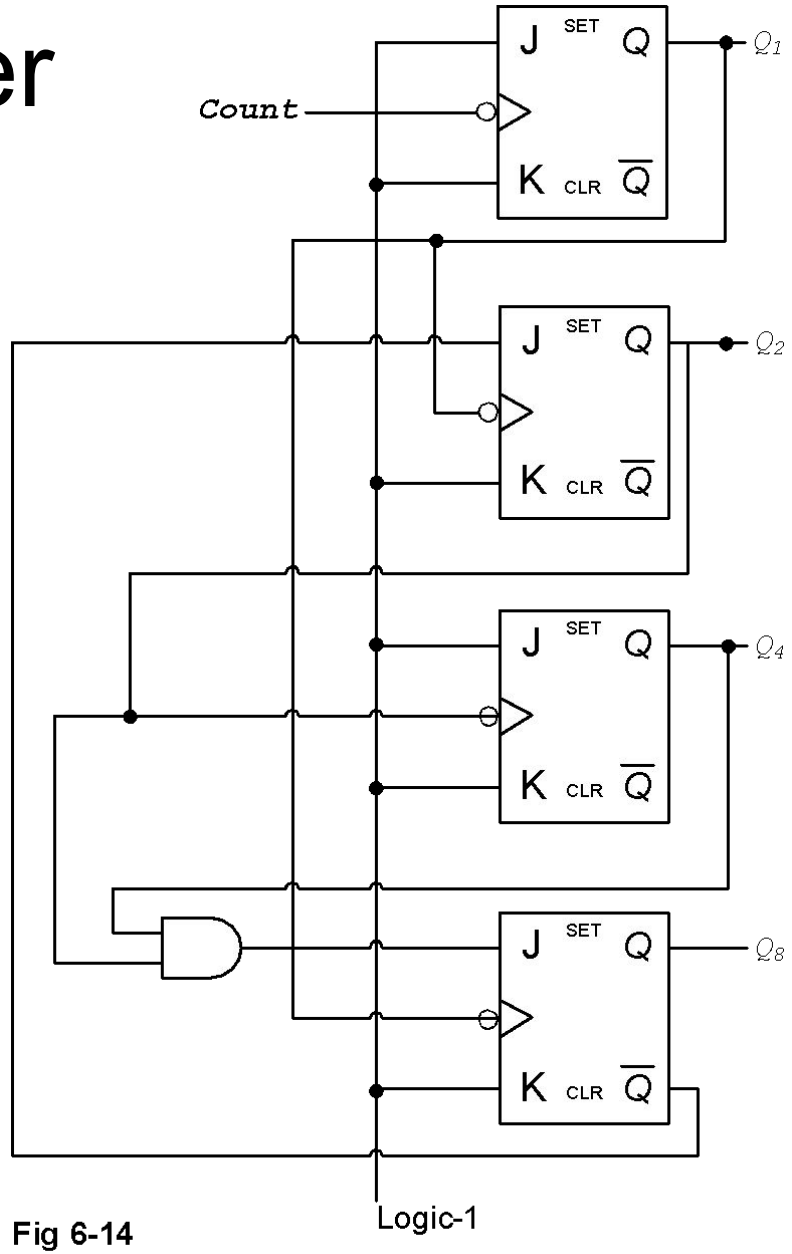
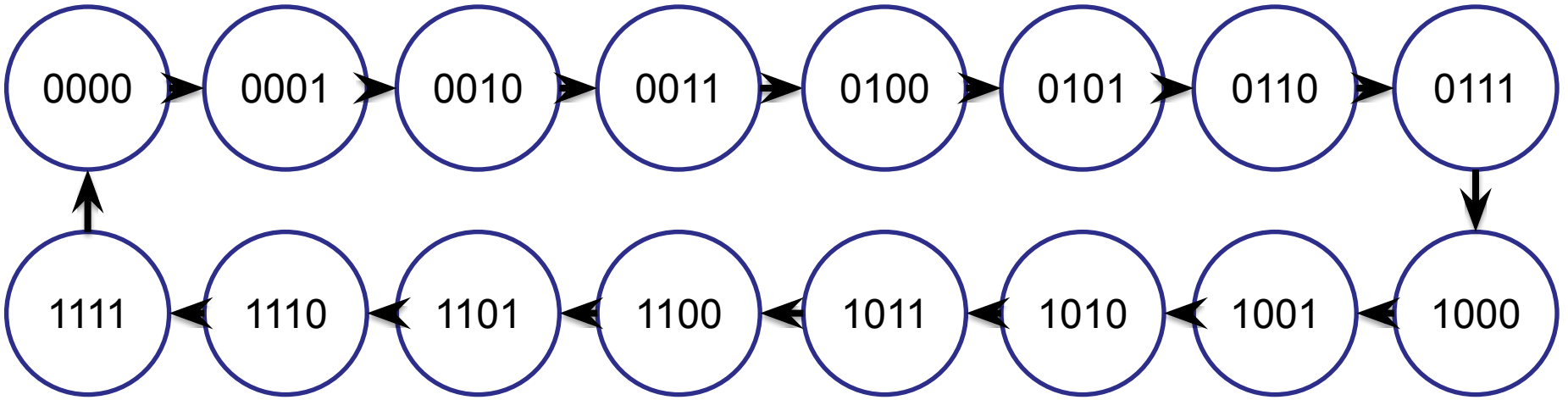


Fig 6-14
BCD ripple counter

4-bit Binary Synchronous Counter



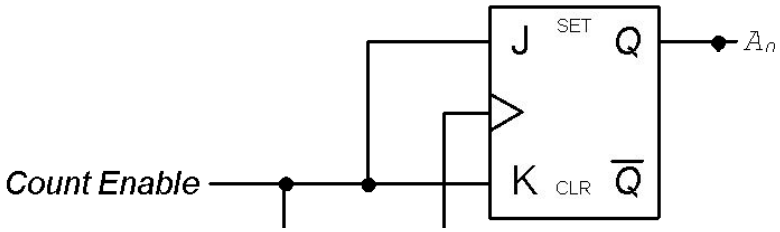
4-bit Binary Synchronous Counter

Present State					Next State			
A3	A2	A1	A0		A3	A2	A1	A0
0	0	0	0		0	0	0	1
0	0	0	1		0	0	1	0
0	0	1	0		0	0	1	1
0	0	1	1		0	1	0	0
0	1	0	0		0	1	0	1
0	1	0	1		0	1	1	0
0	1	1	0		0	1	1	1
0	1	1	1		1	0	0	0
1	0	0	0		1	0	0	1
1	0	0	1		1	0	1	0
1	0	1	0		1	0	1	1
1	0	1	1		1	1	0	0
1	1	0	0		1	1	0	1
1	1	0	1		1	1	1	0
1	1	1	0		1	1	1	1
1	1	1	1		0	0	0	0

4-bit Binary Synchronous Counter

Present State				Next State				Inputs to Flip Flops							
A3	A2	A1	A0	A3	A2	A1	A0	J3	K3	J2	K2	J1	K1	J0	K0
0	0	0	0	0	0	0	1	0	X	0	X	0	X	1	X
0	0	0	1	0	0	1	0	0	X	0	X	1	X	X	1
0	0	1	0	0	0	1	1	0	X	0	X	X	0	1	X
0	0	1	1	0	1	0	0	0	X	1	X	X	1	X	1
0	1	0	0	0	1	0	1	0	X	X	0	0	X	1	X
0	1	0	1	0	1	1	0	0	X	X	0	1	X	X	1
0	1	1	0	0	1	1	1	0	X	X	0	X	0	1	X
0	1	1	1	1	0	0	0	1	X	X	1	X	1	X	1
1	0	0	0	1	0	0	1	X	0	0	X	0	X	1	X
1	0	0	1	1	0	1	0	X	0	0	X	1	X	X	1
1	0	1	0	1	0	1	1	X	0	0	X	X	0	1	X
1	0	1	1	1	1	0	0	X	0	1	X	X	1	X	1
1	1	0	0	1	1	0	1	X	0	X	0	0	X	1	X
1	1	0	1	1	1	1	0	X	0	X	0	1	X	X	1
1	1	1	0	1	1	1	1	X	0	X	0	X	0	1	X
1	1	1	1	0	0	0	0	X	1	X	1	X	1	X	1

4-bit Binary Synchronous Counter

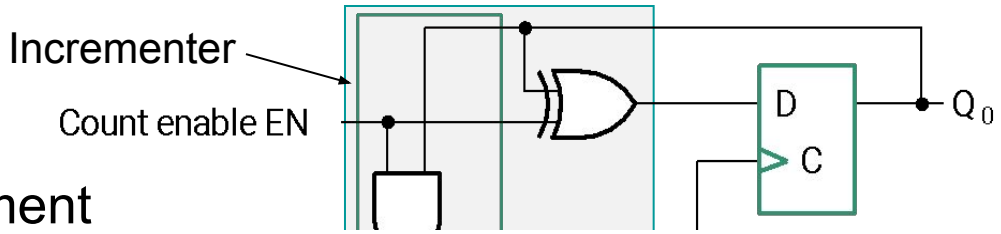


A3	A2	A1	A0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1
0	0	0	0

Fig
4-bi

Synchronous Counters (continued)

- Internal Logic
  - XOR complements each bit
  - AND chain causes complement of a bit if all bits toward LSB from it equal 1
- Count Enable
  - Forces all outputs of AND chain to 0 to “hold” the state
- Carry Out
  - Added as part of incrementer
  - Connect to Count Enable of additional 4-bit counters to form larger counters

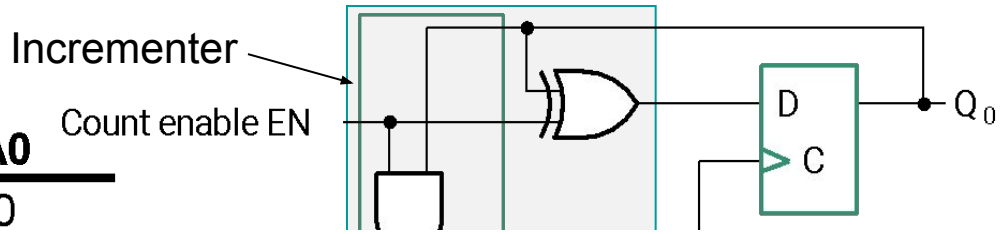


t CO

Cloc

Synchronous Counters (continued)

A3	A2	A1	A0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1
0	0	0	0



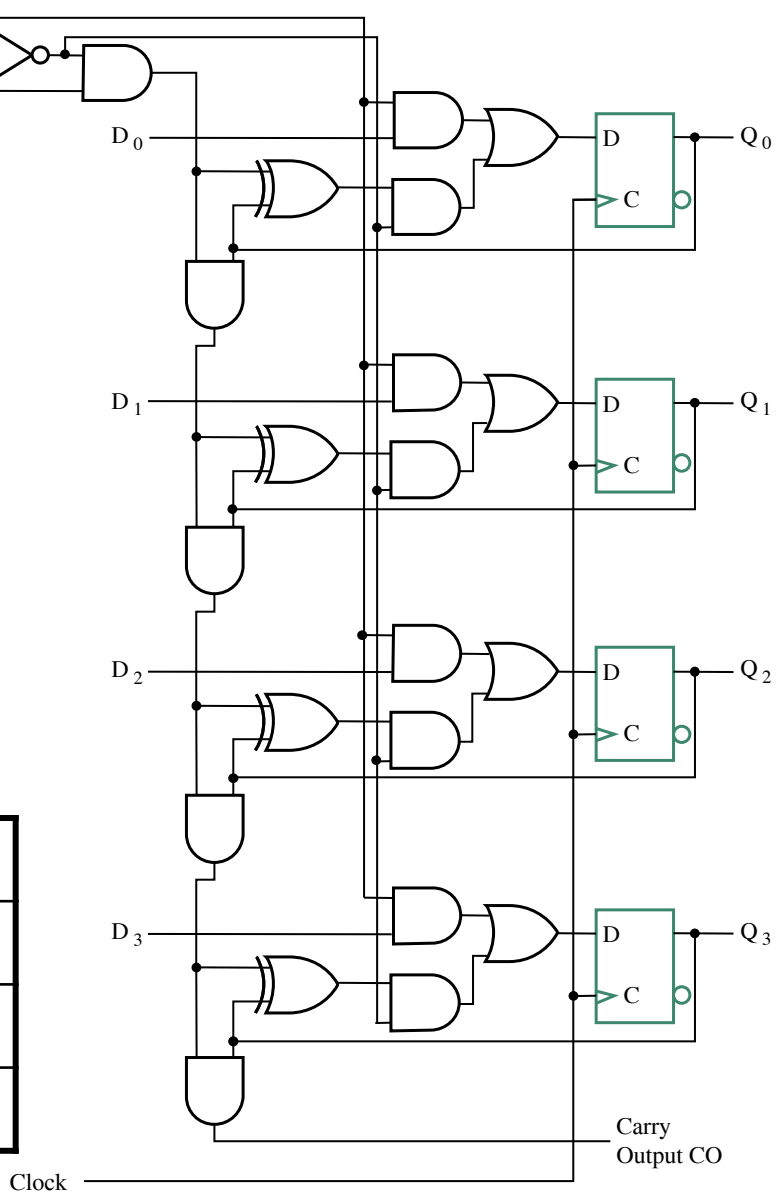
Cloc

t CO

Counter with Parallel Load

- Add path for input data
  - enabled for Load = 1
- Add logic to:
  - disable count logic for Load = 1
  - disable feedback from outputs for Load = 1
  - enable count logic for Load = 0 and Count = 1
- The resulting function table:

Load	Count	Action
0	0	Hold Stored Value
0	1	Count Up Stored Value
1	X	Load D



Design Example: Synchronous BCD

- Use the sequential logic model to design a synchronous BCD counter with D flip-flops
- State Table =>
- Input combinations 1010 through 1111 are don't cares

Current State				Next State			
Q8	Q4	Q2	Q1	Q8	Q4	Q2	Q1
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	0	0	0	0

Synchronous BCD (continued)

- Use K-Maps to two-level optimize the next state equations and manipulate into forms containing XOR gates:

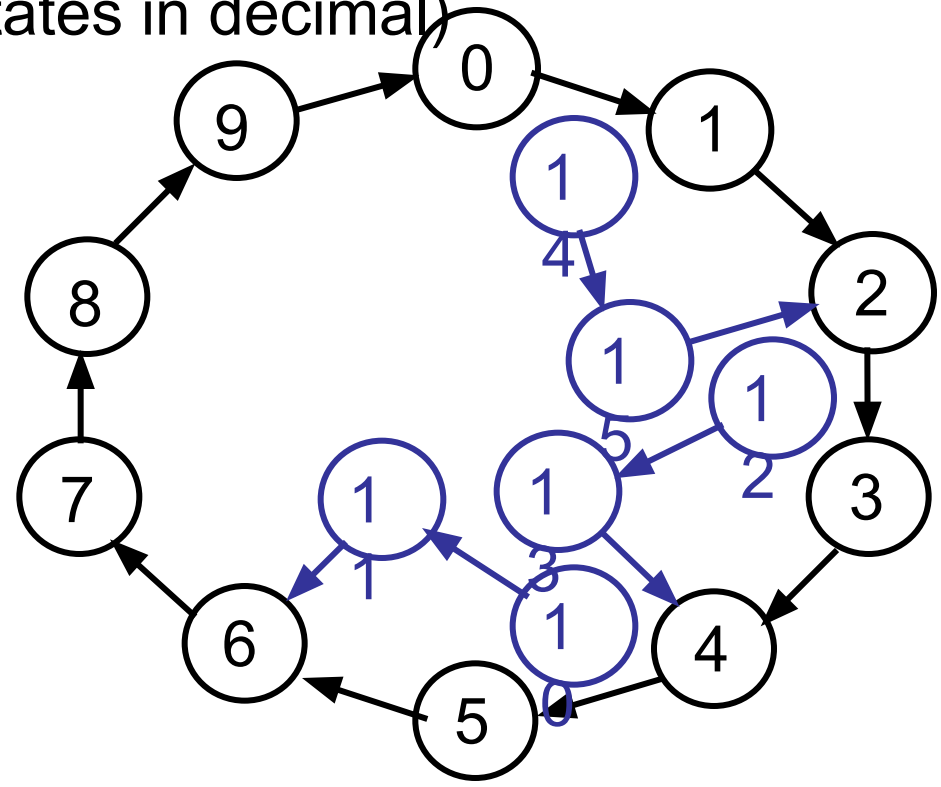
$$D1 = \overline{Q1}$$
$$D2 = Q2 + Q1 \oplus Q8$$
$$D4 = Q4 + Q1 \oplus Q2$$
$$D8 = Q8 + (Q1 \oplus Q8 + Q1Q2Q4)$$

- The logic diagram can be draw from these equations
  - An asynchronous or synchronous reset should be added
- What happens if the counter is perturbed by a power disturbance or other interference and it enters a state other than 0000 through 1001?

Synchronous BCD (continued)

- Find the actual values of the six next states for the don't care combinations from the equations
- Find the overall state diagram to assess behavior for the don't care states (states in decimal)

Present State				Next State			
Q8	Q4	Q2	Q1	Q8	Q4	Q2	Q1
1	0	1	0	1	0	1	1
1	0	1	1	0	1	1	0
1	1	0	0	1	1	0	1
1	1	0	1	0	1	0	0
1	1	1	0	1	1	1	1
1	1	1	1	0	0	1	0

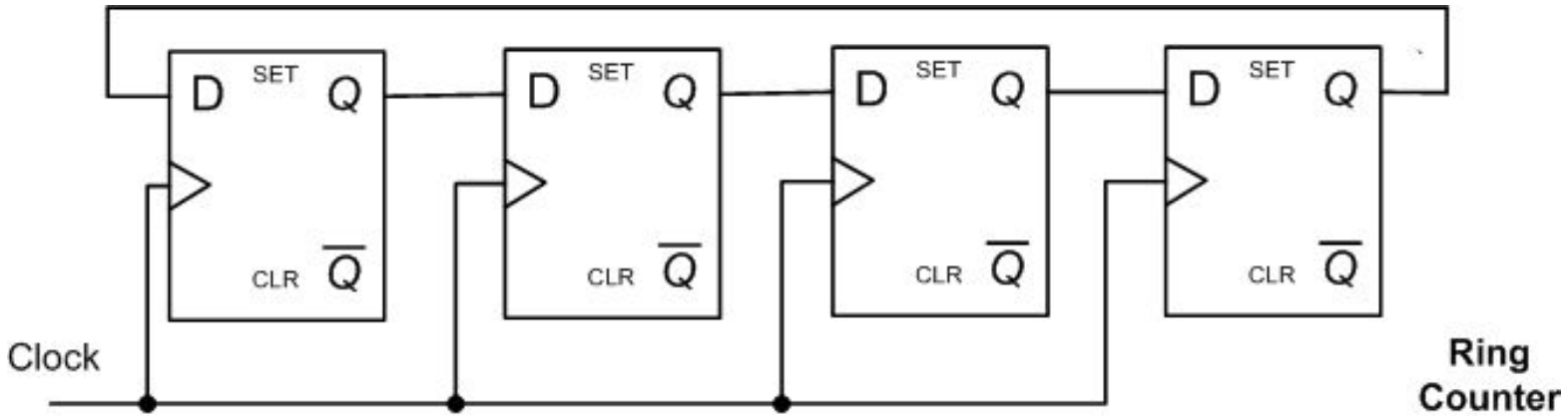


Synchronous BCD (continued)

- For the BCD counter design, if an invalid state is entered, return to a valid state occurs within two clock cycles
- Is this adequate? If not:
  - Is a signal needed that indicates that an invalid state has been entered? What is the equation for such a signal?
  - Does the design need to be modified to return from an invalid state to a valid state in one clock cycle?
  - Does the design need to be modified to return from a invalid state to a specific state (such as 0)?
- The action to be taken depends on:
  - the application of the circuit
  - design group policy

Ring Counter

Clock	Q0	Q1	Q2	Q3
0	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1
4	1	0	0	0



Johnson Counter

Clock	Q0	Q1	Q2	Q3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1
5	0	1	1	1
6	0	0	1	1